```
                    break;
                case F4:
                    if( control !=0 )
                        printf("\n Tích a * b =%d", a * b);
                    break;
                case F5:
                    if( control !=0 )
                     printf("\nThương a*b=%6.2f",(float)a/ (float)b);
                    break;
            }
            clrscr();
        } while(key!=F10);
    }
```

3.7. Lệnh break và lệnh Continue

Lệnh break cho phép ra khỏi các chu trình với các toán tử for, while và switch.

Ví dụ 1: Tính tổng S =1+2+3+4+5

```
s=0;
i=1;
for(i=1;i<=10;i++)
 {if(i==6) break;
   s=s+i;
}
```

Lệnh continue dùng để bắt đầu một vòng mới của chu trình chứa nó. Lệnh continue trong trường hợp for điều khiển được chuyển về bước khởi đầu lại. Còn trong while, do...while quyền điều khiển chương trình được chuyển đến bước kiểm tra điều kiện.

Ví dụ 2: Tính tổng S =1+2+3+4+5+7+8+9+10

```
s=0;
i=1;
for(i=1;i<=10;i++)
{if(i==6) continue;
   s=s+i;
}
```

ở ví dụ 1 S= 1+2+3+4+5 = 15

ở ví dụ 2 S= 1+2+3+4+5+7+8+9+10 = 49

4. HÀM VÀ PHẠM VI HOẠT ĐỘNG CỦA BIẾN

4.1. Tính chất của hàm

Hàm (function) hay nói đúng hơn là chương trình con (sub_program) chia cắt các nhiệm vụ tính toán lớn thành các công việc nhỏ hơn và có thể sử dụng nó ở mọi lúc trong chương trình, đồng thời hàm cũng có thể được cung cấp cho nhiều người khác sử dụng dưới dạng thư viện mà không cần phải bắt đầu xây dựng lại từ đầu. Các hàm thích hợp còn

có thể che dấu những chi tiết thực hiện đối với các phần khác trong chương trình, vì những phần này không cần biết hàm đó thực hiện như thế nào.

Một chương trình C nói chung bao gồm nhiều hàm nhỏ chứ không phải là một vài hàm lớn. Chương trình có thể nằm trên một hoặc nhiều tệp gốc theo mọi cách thuận tiện; các tệp gốc có thể được dịch tách bạch và nạp vào cùng nhau, cùng với các hàm đã được dịch từ trước trong thư viện. Sau đây là một số tính chất cơ bản của hàm:

- Hàm có thể có kiểu hoặc vô kiểu, kiểu ở đây được hiểu là kiểu giá trị trả về của hàm. Kiểu giá trị trả về của hàm có thể là kiểu cơ bản (base type) hoặc có kiểu do người dùng định nghĩa (user type). Trong trường hợp hàm vô kiểu, C sử dụng từ khoá void để chỉ lớp các hàm kiểu này.

- Hàm có thể có tham số hoặc không có tham số. Trong trường hợp hàm không có tham số C sử dụng từ khoá void để chỉ lớp hàm dạng này . Một lời gọi hàm có nghĩa khi và chỉ khi hàm nhận được đầy đủ giá trị các tham số của nó một cách tường minh.

- Giá trị trả về của hàm được thực hiện bằng lệnh return(giá_trị), giá trị trả về của hàm phải phù hợp với kiểu của hàm. Trong trường hợp hàm vô kiểu, ta có thể sử dụng lệnh return hoặc bỏ qua lệnh return;

- Hàm có thể làm thay đổi nội dung của biến hoặc không làm thay đổi nội dung của biến được truyền cho hàm từ chương trình chính. Nếu ta truyền cho hàm là địa chỉ của biến thì mọi thao tác đối với biến trong hàm đều có thể dẫn tới sự thay đổi nội dung của biến trong chương trình chính, cơ chế này được gọi là cơ chế truyền tham biến cho hàm. Nếu ta truyền cho hàm là nội dung của biến thì mọi sự thay đổi nội dung của biến trong hàm không dẫn tới sự thay đổi nội dung của biến trong chương trình chính, cơ chế này dược gọi là cơ chế truyền tham trị.

4.2. Khai báo, thiết kế hàm

Mọi hàm trong C dù là nhỏ nhất cũng phải được thiết kế theo nguyên tắc sau:

```
Kiểu_hàm  Tên_hàm ( Kiểu_1 biến_1, Kiểu_2 biến_2, . . .)
{    Khai báo biến cục bộ trong hàm;
     Câu_lệnh_hoặc_dãy_câu_lệnh;
     return(giá_trị);
}
```

Ghi chú: Trước khi sử dụng hàm cần phải khai báo nguyên mẫu cho hàm (function prototype) và hàm phải phù hợp với nguyên mẫu của chính nó. Nguyên mẫu của hàm thường được khai báo ở phần đầu chương trình theo cú pháp như sau:

```
Kiểu_hàm  Tên_hàm ( Kiểu_1, Kiểu_2 , . . .);
```

Ví dụ: Viết chương trình tìm USCLN của hai số nguyên dương a, b.

```
/* Ví dụ về hàm trả lại một số nguyên int*/
```

```
#include    <stdio.h>
#include    <conio.h>
/* khai báo nguyên mẫu cho hàm; ở đây hàm USCLN trả lại một số nguyên và có hai biến kiểu nguyên */
int   USCLN( int , int ); /* mô tả hàm */
int USCLN( int     a, int    b)
{     while(a!=b){
            if ( a >b )       a = a -b;
            else     b = b-a;          }
      return(a);}
/* chương trình chính */
void main(void) {
      unsigned int    a, b;     clrscr();
      printf("\n Nhập a ="); scanf("%d", &a);
      printf("\n Nhập b ="); scanf("%d", &b);
      printf("\n Ước số chung lớn nhất : %d",USCLN(a,b));
      getch();}
```

Ví dụ: Viết hàm chuyển đổi kí tự in hoa thành kí tự in thường.

```
/* Ví dụ về hàm trả lại một kí tự*/
#include    <stdio.h>
#include    <conio.h>
/* khai báo nguyên mẫu cho hàm; */
char islower(char);
/* mô tả hàm */
char islower ( char c){
      if(c>='A' && c<='Z')
            c = c + 32;
      return(c);}
/* lời gọi hàm*/
void main(void){
      char c='A';
```

PTIT

```
        printf("\n Kí tự được chuyển đổi : %c", islower(c));
        getch();}
```

Ví dụ: Viết hàm tính luỹ thừa bậc n của số nguyên a.

```
    /* Ví dụ về hàm trả lại một số nguyên dài*/
    #include<stdio.h>
    #include<conio.h>
    /* khai báo nguyên mẫu cho hàm*/
    long power(int , int );
    /* mô tả hàm */
    long power ( int a, int n )
    {     long s =1 ; int i;
    for(i=0; i<n;i++)
    s*=a;
return(s);
       }
    /* lời gọi hàm */
    void main(void)
    {
        int a = 5, i;
        for(i=0; i<50;i++)
                printf("\n %d mũ %d = %ld", a , i, power(a,i);
        getch();
}
```

Ví dụ 4.20: In ra số nhị phân của một số nguyên.

```
    /* Ví dụ về hàm không trả lại giá trị*/
#include<stdio.h>
    #include<conio.h>
    /* khai báo nguyên mẫu cho hàm*/
void binary_int( int );
    /* mô tả hàm */
void binary_int ( int a)
    {
    int i, k=1; clrscr();
    for(i=15; i>=0; i--)
```

PTIT

```
    { if ( a & (k<<i))
          printf("%3d", 1);
else
         printf("%3d", 0);
          }
}
    /* lời gọi hàm */
    void main(void)
    {
          int a;
          printf("\n Nhập a="); scanf("%d", &a);
          printf("\n Số nhị phân của %d:", a);
          binary_int(a);
    getch();
}
```

4.3. Phương pháp truyền tham biến cho hàm

Để thấy rõ được hai phương pháp truyền tham trị và truyền tham biến của hàm chúng ta khảo sát ví dụ sau:

Ví dụ: Cho hai số a, b hãy viết hàm đổi chỗ hai số a và b.

```
    /* Phương pháp truyền tham trị */
    #include<stdio.h>
void swap( float , float );
    void swap ( float a, float b)
    {
    float temp;
          temp = a;
          a = b;
          b = temp;
}
void main(void)
{
    float a = 5, b = 7;
          swap(a, b);
```

PTIT

```
    /* thực hiện đổi chỗ */
    printf("\n Giá trị a = %6.2f, b =%6.2f", a, b);
}
```

Kết quả thực hiện :

Giá trị của a = 5, b = 7

Nhận xét: Hai biến a, b không được hoán vị cho nhau sau khi thực hiện hàm swap(a,b). Lý do duy nhất để dẫn đến sự kiện này là hàm swap(a,b) thực hiện trên bản sao giá trị của biến a và b. Phương pháp truyền giá trị của biến cho hàm được gọi là phương pháp truyền theo tham trị. Nếu muốn a, b thực sự hoán vị nội dung cho nhau chúng ta phải truyền cho hàm swap(a, b) địa chỉ của ô nhớ của a và địa chỉ ô nhớ của b. Khi đó các thao tác hoán đổi nội dung biến a và b được xử lý trong hàm swap(a, b) thực chất là hoán đổi nội dung của ô nhớ dành cho a thành nội dung ô nhớ dành cho b và ngược lại.

Ví dụ sau sẽ minh hoạ cơ chế truyền tham biến cho hàm, trước khi chúng ta chưa thảo luận kỹ về con trỏ (pointer), ta tạm ngầm hiểu các qui định như sau:

Toán tử : &(tên_biến) dùng để lấy địa chỉ của biến , chính xác hơn là địa chỉ ô nhớ dành cho biến.

Toán tử : *(tên_biến) dùng để lấy nội dung của ô nhớ dành cho biến.

Ví dụ: Cho hai số a, b hãy viết hàm đổi chỗ hai số a và b.

```
    /* Phương pháp truyền tham trị */
    #include    <stdio.h>
void swap( float , float );
void swap ( float *a, float *b)
    {   float temp;
        temp = *a;
        *a = *b;
        *b = temp;
}
void main(void)
    {   float   a = 5, b = 7;
        swap(&a, &b); /* thực hiện đổi chỗ trên địa chỉ của a và địa chỉ của b*/
        printf("\n Giá trị a = %6.2f    b =%6.2f", a, b);
}
```

Kết quả thực hiện :

Giá trị của a = 7 b = 5

Nhận xét: Giá trị của biến bị thay đổi sau khi hàm swap() thực hiện trên địa chỉ của hai biến a và b. Cơ chế truyền cho hàm theo địa chỉ của biến được gọi là phương pháp truyền tham biến cho hàm. Nếu hàm được truyền theo tham biến thì nội dung của biến sẽ bị thay đổi sau khi thực hiện hàm.

4.4. Biến địa phương, biến toàn cục

a) Biến toàn cục

Biến toàn cục là biến được khai báo ở ngoài tất cả các hàm (kể cả hàm main()). Vùng bộ nhớ cấp phát cho biến toàn cục được xác định ngay từ khi kết nối (link) và không bị thay đổi trong suốt thời gian chương trình hoạt động. Cơ chế cấp phát bộ nhớ cho biến ngay từ khi kết nối còn được gọi là cơ chế cấp phát tĩnh.

Nội dung của biến toàn cục luôn bị thay đổi theo mỗi thao tác xử lý biến toàn cục trong chương trình con, do vậy khi sử dụng biến toàn cục ta phải quản lý chặt chẽ sự thay đổi nội dung của biến trong chương trình con.

Phạm vi hoạt động của biến toàn cục được tính từ vị trí khai báo nó cho tới cuối văn bản chương trình. Về nguyên tắc, biến toàn cục có thể khai báo ở bất kỳ vị trí nào trong chương trình, nhưng nên khai báo tất cả các biến toàn cục lên đầu chương trình vì nó làm cho chương trình trở nên sáng sủa và dễ đọc, dễ nhìn, dễ quản lý.

Ví dụ: Ví dụ về biến toàn cục

```
/* Ví dụ về biến toàn cục*/
#include     <stdio.h>
#include     <conio.h>
/* khai báo nguyên mẫu cho hàm*/
void Tong_int( void );
/* khai báo biến toàn cục*/
 int  a = 5, b=7;
/* mô tả hàm */
int tong(void)
{     printf("\n Nhap a="); scanf("%d",&a);
      printf("\n Nhap b="); scanf("%d",&b);
      return(a+b);}
/* chương trình chính */
void main(void){
      printf("\n Giá trị a, b trước khi thực hiện hàm ");
      printf(" a =%5d b = %5d a + b =%5d", a, b, a + b);
      printf("\n Giá trị a, b sau khi thực hiện hàm ");
      printf(" a =%5d b = %5d a + b =%5d", a, b,  tong());
}
```

Kết quả thực hiện:

Giá trị a, b trước khi thực hiện hàm a =5 b = 7 a + b = 12

Giá trị a, b sau khi thực hiện hàm

PTIT

Nhập a = 10
Nhập b = 20
a = 10 b = 20 a + b = 30

b) Biến địa phương

Biến địa phương là các biến được khai báo trong các hàm và chỉ tồn tại trong thời gian hàm hoạt động. Tầm tác dụng của biến địa phương cũng chỉ hạn chế trong hàm mà nó được khai báo, không có mối liên hệ nào giữa biến toàn cục và biến địa phương mặc dù biến địa phương có cùng tên, cùng kiểu với biến toàn cục.

Cơ chế cấp phát không gian nhớ cho các biến địa phương được thực hiện một cách tự động, khi nào khởi động hàm thì các biến địa phương được cấp phát bộ nhớ. Mỗi lần khởi động hàm là một lần cấp phát bộ nhớ, do vậy địa chỉ bộ nhớ dành cho các biến địa phương luôn luôn thay đổi sau mỗi lần gọi tới hàm.

Nội dung của các biến địa phương không được lưu trữ sau khi hàm thực hiện, các biến địa phương sinh ra sau mỗi lần gọi hàm và bị giải phóng ngay sau khi ra khỏi hàm. Các tham số của hàm cũng là biến địa phương. Nghĩa là, tham số của hàm cũng chỉ được khởi động khi gọi tới hàm.

Biến địa phương tĩnh (static): là biến địa phương đặc biệt được khai báo thêm bởi từ khoá static. Khi một biến địa phương được khai báo là static thì biến địa phương được cấp phát một vùng bộ nhớ cố định vì vậy nội dung của biến địa phương sẽ được lưu trữ lại lần sau và tồn tại ngay cả khi hàm đã kết thúc hoạt động. Mặc dù biến toàn cục và biến địa phương tồn tại trong suốt thời gian chương trình hoạt động nhưng điều khác nhau cơ bản giữa chúng là biến toàn cục có thể được truy nhập và sử dụng ở mọi lúc, mọi nơi, còn biến địa phương static chỉ có tầm hoạt động trong hàm mà nó được khai báo là static.

Ví dụ: Ví dụ về sử dụng biến địa phương static trong hàm

bien_static() chứa biến tĩnh i và kiểm tra nội dung của i sau 5 lần gọi tới hàm.

```
#include<stdio.h>

/* nguyên mẫu của hàm */

void bien_static(void);

/* mô tả hàm */
void bien_static(void) {

static int i;-   /* khai báo biến static */

    i++;

    printf("\n Lần gọi thứ %d", i);

}

void main(void){
    int n;

    for(n=1; n<=5; n++)
```

```
        bien_static();
}
```

Kết quả thực hiện:

Lần gọi thứ 1

Lần gọi thứ 2

Lần gọi thứ 3
Lần gọi thứ 4
Lần gọi thứ 5

Biến địa phương dạng thanh ghi (register) **:** Chúng ta đã biết rằng các bộ vi xử lý đều có các thanh ghi, các thanh ghi nằm ngay trong CPU và không có địa chỉ riêng biệt như các ô nhớ khác trong bộ nhớ chính nên tốc độ xử lý cực nhanh. Do vậy, để tận dụng ưu điểm về tốc độ của các thanh ghi chúng ta có thể khai báo một biến địa phương có kiểu register. Tuy nhiên, việc làm này cũng nên hạn chế vì số thanh ghi tự do không có nhiều. Nên sử dụng biến thanh ghi trong các trường hợp biến đó là biến đếm trong các vòng lặp.

Ví dụ: Biến địa phương có sử dụng register.

```
#include<stdio.h>

/* nguyên mẫu của hàm */
void bien_static(void);

/* mô tả hàm */
void bien_static(void) {
    static int i;

    /* khai báo biến static */
    i++;
    printf("\n Lần gọi thứ %d", i);
}

void main(void){

    register int n;

    for(n=1; n<=5; n++)
        bien_static();
}
```

Kết quả thực hiện

Lần gọi thứ 1

Lần gọi thứ 2

Lần gọi thứ 3

Lần gọi thứ 4

PTIT

Lần gọi thứ 5

4.5. *Tính đệ qui của hàm*

Một lời gọi hàm được gọi là đệ qui nếu nó gọi đến chính nó. Tính đệ qui của hàm cũng giống như phương pháp định nghĩa đệ qui của qui nạp toán học, hiểu rõ được tính đệ qui của hàm cho phép ta cài đặt rộng rãi lớp các hàm toán học được định nghĩa bằng đệ qui và giảm thiểu quá trình cài đặt chương trình.

Ví dụ: Nhận xét và cài đặt hàm tính n! của toán học

n ! = 1 khi n=0;

(n-1)! * n khi n>=1;

```
/* chương trình tính n! bằng phương pháp đệ qui */
#include<stdio.h>
#include<conio.h>
/* khai báo nguyên mẫu của hàm */
unsigned long GIAI_THUA( unsigned int );
/* mô tả hàm */
unsigned long GIAI_THUA(unsigned int n){
    if (n = = 0)
        return(1);
    else
        return ( n * GIAI_THUA(n-1));
}
void main(void) {
    unsigned int n;
    printf("\ Nhập n ="); scanf("%d", &n);
    printf("\n n! = %ld", GIAI_THUA(n));
}
```

Ghi chú: Việc làm đệ qui của hàm cần sử dụng bộ nhớ theo kiểu xếp chồng LIFO (Last In, First Out để chứa các kết quả trung gian, do vậy việc xác định điểm kết thúc quá trình gọi đệ qui là hết sức quan trọng. Nếu không xác định rõ điểm kết thúc của quá trình chương trình sẽ bị treo vì lỗi tràn stack (stack overflow).

Ví dụ:

Viết đệ qui hàm tìm ước số chung lớn nhất của hai số nguyên dương a, b.

```
int USCLN( int a, int b){
if( b = = 0) return(a);
```

PTIT

```
    else return( USCLN( b, a %b));
}
```

Ví dụ: Giải quyết bài toán kinh điển trong các tài liệu về ngôn ngữ lập trình "bài toán Tháp Hà Nội ". Bài toán được phát biểu như sau:

Có ba cột C1, C2, C3 dùng để xếp đĩa theo thứ tự đường kính giảm dần của các chiếc đĩa. Hãy tìm biện pháp dịch chuyển N chiếc đĩa từ cột C1 sang cột C2 sao cho các điều kiện sau được thoả mãn:

- Mỗi lần chỉ được phép dịch chuyển một đĩa
- Mỗi đĩa có thể được dịch chuyển từ cột này sang một cột khác bất kỳ
- Không được phép để một đĩa trên một đĩa khác có đường kính nhỏ hơn

Ta nhận thấy, với N = 2 chúng ta có cách làm như sau: Chuyển đĩa bé nhất (đĩa 1) sang C3, chuyển đĩa còn lại sang C2, chuyển đĩa 1 từ C3 sang C2.

Với N = 3 ta lại xử lý lần lượt như sau với giả thiết đã biết cách làm với N = 2 (N - 1 đĩa): Chuyển đĩa 1, 2 sang C3 theo như cách làm với N=2; chuyển đĩa 3 sang cột 2, chuyển đĩa 1 và 2 từ C3 sang C2.

Chúng ta có thể tổng quát hoá phương pháp dịch chuyển bằng hàm sau:

DICH_CHUYEN(N_đĩa, Từ_Cột, Đến_Cột, Cột_Trung_Gian);

Với N=2 công việc có thể được diễn tả như sau:

DICH_CHUYEN(1, C1, C3 , C2);

DICH_CHUYEN(1, C1, C2 , C3);

DICH_CHUYEN(1, C3, C2 , C1);

Với N=3 công việc dịch chuyển thực hiện như N =2 nhưng thực hiện dịch chuyển 2 đĩa

DICH_CHUYEN(2, C1, C3 , C2);

DICH_CHUYEN(1, C1, C2 , C3);

DICH_CHUYEN(2, C3, C2 , C1);

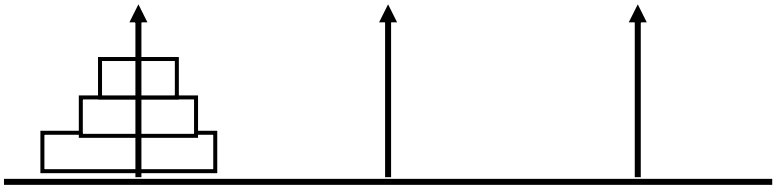
Với N tổng quát ta có :

DICH_CHUYEN(N - 1, C1, C3 , C2);

DICH_CHUYEN(1, C1, C2 , C3);

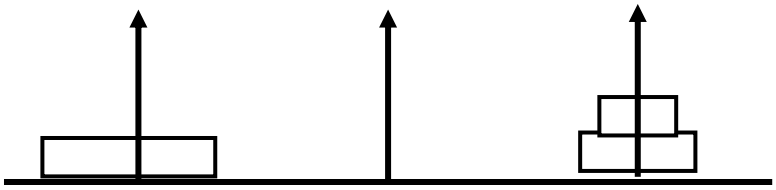
DICH_CHUYEN(N - 1 , C3, C2 , C1);

Yêu cầu ban đầu: dịch chuyển N đĩa từ cột C1 sang cột C2 thông qua cột trung gian C3:



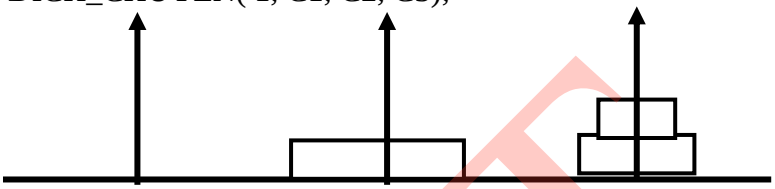
C1 C2 C3

Thực hiện: DICH_CHUYEN(N-1, C1, C3, C2);



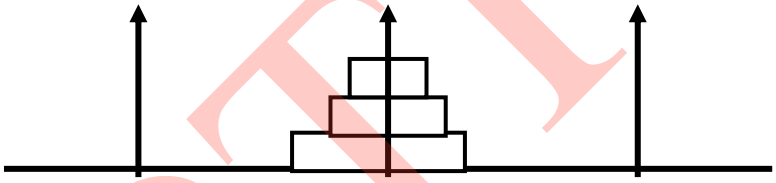
C1 C2 C3

Thực hiện: DICH_CHUYEN(1, C1, C2, C3);



C1 C2 C3

Thực hiện: DICH_CHUYEN(N-1, C3, C2, C1);



C1 C2 C3

PTIT

Bài toán Tháp Hà Nội được thể hiện thông qua đoạn chương trình sau:

```
#include     <stdio.h>
#include     <conio.h>
/* Khai báo nguyên mẫu cho hàm*/
voidDICH_CHUYEN (int , int , int , int );
/* Mô tả hàm */
voidDICH_CHUYEN (int N, int C1, int C2, int C3) {
      if ( N ==1 )
printf("\n %5d -> %5d", C1, C2);
else {
                DICH_CHUYEN ( N-1, C1, C3, C2);
                DICH_CHUYEN ( 1, C1, C2, C3);
                DICH_CHUYEN ( N-1, C3, C2, C1);
```

```
        }
}
```

5. CẤU TRÚC DỮ LIỆU KIỂU MẢNG (Array)

5.1. Khái niệm về mảng

Mảng là một tập cố định các phần tử cùng có chung một kiểu dữ liệu với các thao tác tạo lập mảng, tìm kiếm, truy cập một phần tử của mảng, lưu trữ mảng. Ngoài giá trị, mỗi phần tử của mảng còn được đặc trưng bởi chỉ số của nó thể hiện thứ tự của phần tử đó trong mảng. Không có các thao tác bổ sung thêm vùng nhớ hoặc loại bỏ vùng nhớ của mảng vì số vùng nhớ cho phần tử trong mảng là cố định.

Một mảng một chiều gồm n phần tử được coi như một vector n thành phần, phần tử thứ i của nó được tương ứng với một chỉ số thứ i - 1 đối với ngôn ngữ lập trình C vì phần tử đầu tiên được bắt đầu từ chỉ số 0. Chúng ta có thể mở rộng khái niệm của mảng một chiều thành khái niệm về mảng nhiều chiều.

Một mảng một chiều gồm n phần tử trong đó mỗi phần tử của nó lại là một mảng một chiều gồm m phần tử được gọi là một mảng hai chiều gồm n x m phần tử.

Tổng quát, một mảng gồm n phần tử mà mỗi phần tử của nó lại là một mảng k - 1 chiều thì nó được gọi là mảng k chiều. Số phần tử của mảng k chiều là tích số giữa số các phần tử của mỗi mảng một chiều.

Khai báo mảmg một chiều được thực hiện theo qui tắc như sau:

```
Tên_kiểu      Tên_biến[Số_phần tử];
```

Ví dụ :

```
int     A[10];  /* khai báo mảng tối đa chứa 10 phần tử nguyên*/
char    str[20]; /* khai báo mảng tối đa chứa 19 kí tự */
float   B[20];  /* khai báo mảng tối đa chứa 20 số thực */
long    int L[20]; /* khai báo mảng tối đa chứa 20 số nguyên dài */
```

b- Cấu trúc lưu trữ của mảng một chiều

Cấu trúc lưu trữ của mảng: Mảng được tổ chức trong bộ nhớ như một vector, mỗi thành phần của vector được tương ứng với một ô nhớ có kích cỡ đúng bằng kích cỡ của kiểu phần tử và được lưu trữ kế tiếp nhau. Nếu chúng ta có khai báo mảng gồm n phần tử thì phần tử đầu tiên là phần tử thứ 0 và phần tử cuối cùng là phần tử thứ n - 1, đồng thời mảng được cấp phát một vùng không gian nhớ liên tục có số byte được tính theo công thức:

Kích_cỡ_mảng = (Số_phần_tử * sizeof (kiểu_phần_tử).

Ví dụ chúng ta có khai báo:

```
int   A[10];
```

Khi đó kích cỡ tính theo byte của mảng là :